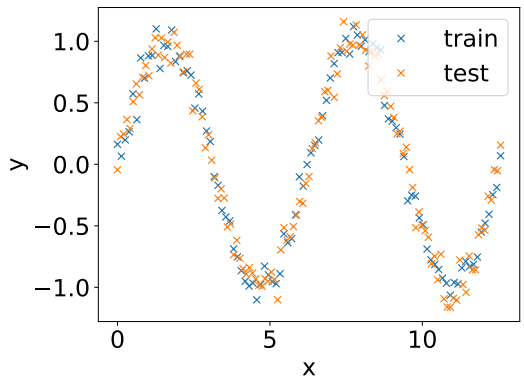
======= TODO Student 2 name (login, matriculation number) ¿¿¿¿¿¿¿ origin/tu

DL26, Assignment 1

Anh Tu Duong Nguyen (anguyea, 2115931)
¡¡¡¡¡¡¡ HEAD Anh-Nhat Nguyen (anhnnguy, 2034311)

March 21, 2026

Task 2a)



We conjecture how a fit with ReLU activation would look for different hidden-layer widths:

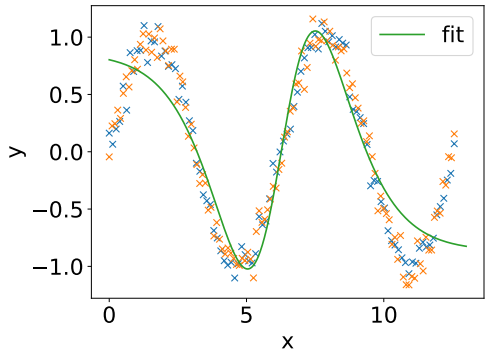
(i) **0 hidden neurons:** The network reduces to linear regression. We expect a flat line through the data mean, unable to capture any non-linearity.

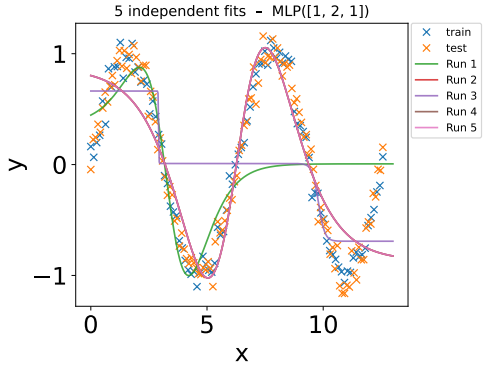
(ii) **1 hidden neuron:** One ReLU unit introduces exactly one bend. We expect a "V" shape that captures at most one slope change.

(iii) **2 hidden neurons:** Two ReLU units allow two bends. We expect an "N" shape that roughly outlines a single peak or valley.

(iv) **3 hidden neurons:** Three ReLU units allow three bends. We expect a "W" shape that tracks the data's up–down–up pattern.

Task 2b)



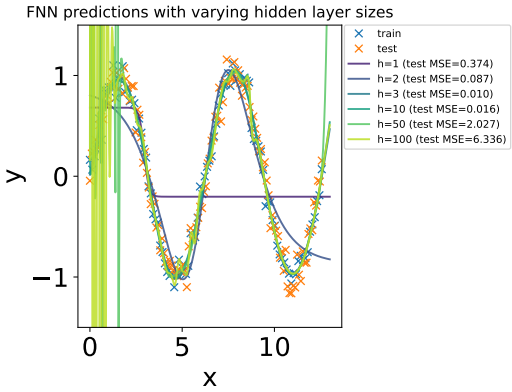


We train a 2-hidden-neuron FNN and obtain a training MSE of 0.0796 and a test MSE of 0.0867.

(i) The result differs from our conjecture: instead of an "N" shape with sharp bends, the fit produces smooth, compressed step-like transitions. This occurs because the code uses sigmoid activation (not ReLU), which generates smooth S-curves rather than piecewise-linear bends.

(ii) Repeating training produces inconsistent fits. Different runs converge to distinct local minima with costs of 0.080, 0.287, and 0.303. Random weight initialization places the optimizer at different starting points in a non-convex loss landscape. With only 2 neurons, the landscape contains multiple local minima, and BFGS converges to whichever basin it starts in.

Task 2c)

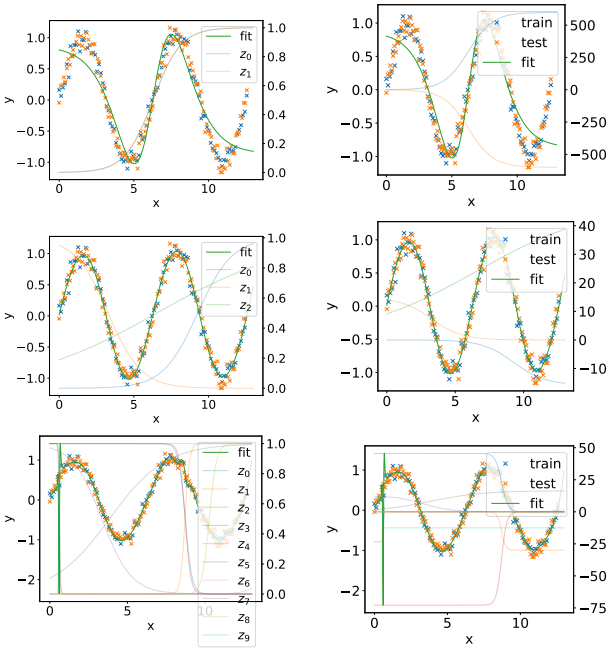


h	Train MSE	Test MSE
1	0.3729	0.3743
2	0.0796	0.0867
3	0.0073	0.0104
10	0.0059	0.0160
50	0.0030	2.0268
100	0.0021	6.3365

(i) As h increases, the model transitions through three phases. *Underfitting* (h=1, 2): the network lacks capacity to capture the wave pattern. *Optimal* (h=3): both MSEs reach low values (0.0073 and 0.0104), indicating the model captures the underlying function without memorizing noise. *Overfitting* ($h \geq 10$): training MSE continues to decrease, but test MSE diverges sharply (from 0.0160 at h=10 to 6.3365 at h=100) as the network memorizes noise.

(ii) Our conjectures from (a) correctly predicted that h=1 and h=2 underfit and h=3 provides sufficient capacity. We did not anticipate the overfitting at $h \geq 50$, where the model produces chaotic oscillations between training points.

Task 2d)



(i) The FNN obtains flexibility through linear combination of non-linear basis functions. Each hidden neuron produces a sigmoid curve; the output layer scales each by a learned weight and sums them. The scaled plots confirm this: the fit line equals the sum of the scaled hidden outputs.

(ii) The h=2 representation is not intuitive. The unscaled plot shows two monotonic S-curves that reveal little about the fit. The scaled plot exposes extreme cancellation: weights of $\approx \pm\, 547$ produce a peak of height 1.0 by subtracting two large opposing values. This cancellation strategy is mathematically opaque.

(iii) The h=3 model is intuitive: output weights drop to -15 to $+40$, and each neuron maps to a distinct geometric feature (two valleys and one broad peak). The h=10 model abandons this interpretability: multiple neurons overlap or oppose each other, weights inflate to -120 to $+70$, and individual contributions become opaque.

Task 2e)

Optional: Adam optimizer analysis.

We train an h=50 network using the Adam optimizer (learning rate 0.01, 5000 epochs) and compare against BFGS.

Adam converges smoothly from an initial cost of 3.4, decreasing steadily over the first 2000 epochs before plateauing at approximately 0.35. It never reaches the BFGS optimum of 0.003 for the same architecture.

This performance gap reflects the difference between first-order and second-order optimization. Adam uses only gradient information and maintains adaptive per-parameter learning rates. It converges stably but lacks curvature information, so it settles in broad, shallow basins. BFGS approximates the Hessian (second-order) and uses curvature to navigate narrow valleys in the loss landscape, reaching deeper minima in fewer iterations.

For this low-dimensional problem (51 parameters for h=50), BFGS can afford to maintain its $O(n^2)$ Hessian approximation and achieves a superior solution. In high-dimensional settings (thousands of parameters), Adam's $O(n)$ memory cost and mini-batch compatibility make it the practical choice despite its weaker per-step convergence.

Task 3a)

There are 6 quantities computed using our parameters and inputs and targets in this forward pass.

- $z_1 = W_1 x$
- $z_2 = W_2 + b_1$
- $z_3 = \sigma(z_2)$ where $\sigma(x) = \frac{1}{1+e^{-x}}$
- $z_4 = W_2 z_3$
- $\hat{y} = z_4 + b_2$
- $l = (y - \hat{y})^2$

Task 3b)

There are in total 6 gradients to compute in this backward pass. The gradients are computed using backpropagation which is defined as follows:

For a given operator $f : \mathbb{R}^I \to \mathbb{R}^O$, the gradient going outside of the operator back to the input is:

$$\delta_{in} = J_f^T \delta_{out} \text{ where } J_f \text{ is the Jacobian of } f.$$

In the forward pass of the compute graph, the following computations are performed:

- matmul
- logistic activation is applied
- add

Even though the compute graph figure shows that we have linear layers. The inputs to this linear layer are a matrix and a vector. According to the lecture, this equals matmul. The resulting gradient rules are for these operations:

- $\delta_{out} v^T$ or $A^T \delta_{out}$ for the matmul operation
- $\sigma(x)(1 - \sigma(x))\delta_{out}$ for the logistic activation
- δ_{out} for all inputs in the add operation

Applying these rules of the compute graph, we get the following gradients:

- $\delta_l = 1$
- $\delta_{\hat{y}} = -2(y - \hat{y})\delta_l$
- $\delta_y = 2(y - \hat{y})\delta_l$
- $\delta_{z_4} = \delta_{\hat{y}}$
- $\delta_{b_2} = \delta_{\hat{y}}$
- $\delta_{W_2} = \delta_{z_4} z_3^T$
- $\delta_{z_3} = W_2^T \delta_{z_4}$
- $\delta_{z_2} = \sigma(z_2)(1 - \sigma(z_2))\delta_{z_3}$
- $\delta_{b_1} = \delta_{z_2}$
- $\delta_{W_1} = \delta_{z_2} x^T$
- $\delta_x = W_1^T \delta_{z_1}$

Ehrenwörtliche Erklärung

Ich versichere, dass ich die beiliegende Bachelor-, Master-, Seminar-, oder Projektarbeit ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und in der untenstehenden Tabelle angegebenen Hilfsmittel angefertigt und die den benutzten Quellen wörtlich oder inhaltlich entnommenen Stellen als solche kenntlich gemacht habe. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen. Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Declaration of Used AI Tools

Task	Solved with AI?	Tool	Purpose	Useful?
1b	No			
1c	No			
1d	No			
2a	Yes	Claude Code	Revision agent	Yes
		GitHub Copilot	Co-pair coding	Yes
		Google Gemini	Proof check	Yes
2b	Yes	Claude Code	Revision agent	Yes
		GitHub Copilot	Co-pair coding	Yes
		Google Gemini	Proof check	Yes
2c	Yes	Claude Code	Revision agent	Yes
		GitHub Copilot	Co-pair coding	Yes
		Google Gemini	Proof check	Yes
2d	Yes	Claude Code	Revision agent	Yes
		GitHub Copilot	Co-pair coding	Yes
		Google Gemini	Proof check	Yes
3a				
3b				

Unterschrift
Mannheim, den 20.03.2026